

Speaker 2 of 5 — Project Technology Stack

Career Compass SG · Module 1 Assignment · Group 7

Your time: up to 3 minutes · You explain how it is built

Your job in one sentence

Show that the architecture was a choice, not an accident. Anyone can load a CSV. Your three minutes explain *why* we built a three-stage pipeline, and what it bought us: a dashboard that filters a million rows faster than you can blink.

Where you sit in the flow

	Speaker	Topic
	1	Dashboard overview
→ YOU	2	Project technology stack
	3	Data cleaning
	4	Data analysis 1 — Demand vs Competition
	5	Data analysis 2 — Career Recommender

Speaker 1 hands you: "...now, the technology that makes it run." **Your handover to Speaker 3:** "But fast technology means nothing if the data going in is wrong — and this dataset had a trap hidden inside it."

Script

Part 1 — The tools (0:00 – 0:40)

(Slide: the stack logos)

Everything in this project is **Python**. Let me name the five tools and what each one does — one sentence each.

- **pandas** does all the data cleaning and number-crunching.
- **Parquet** is the file format we store clean data in — think of it as a compressed, super-fast version of a spreadsheet file.
- **Streamlit** turns Python code into the web dashboard you just saw.
- **Plotly** draws the interactive charts — hover on anything and you get the details.
- **matplotlib and seaborn** draw the static charts in our notebook and report.

No fancy databases, no cloud services. Simple tools, used carefully.

Part 2 — The three-stage pipeline (0:40 – 1:40)

(Slide: the pipeline diagram)

The important part is not the tools — it is **how we arranged them**. Three stages:

Raw CSV (273 MB, 1.05 million rows) ↓ Stage 1: clean it → takes about 15 seconds Clean Parquet files ↓ Stage 2: pre-compute summaries → takes about 2 seconds Summary tables + scores ↓ Stage 3: the dashboard reads the results

Stage one reads the 273-megabyte raw file in **six chunks of 200,000 rows** — the file is too big to treat carelessly, so we clean it piece by piece. The whole thing runs in about **15 seconds**.

Stage two pre-computes every summary table the dashboard needs.

Stage three is the dashboard itself. The key rule: **the dashboard never touches the raw file**. It only reads the small, clean, pre-processed results. That is why it starts instantly.

Part 3 — The payoff (1:40 – 2:20)

(Slide: the performance numbers)

Here is what this design bought us — one number to remember.

Our working table has **1.77 million rows**. Filtering it and re-computing every chart takes **0.06 seconds**. Six hundredths of a second.

Why does that matter? Because it means our filters are **real**. When you move a slider in our dashboard, we re-calculate everything from the raw rows, live. Many dashboards just reshuffle a frozen summary table. Ours actually recomputes — because we made it fast enough to afford that.

Part 4 — One decision worth defending (2:20 – 2:50)

One last engineering decision. The dashboard **imports its scoring code from the pipeline** — the exact same functions, not a copy.

Why? Because if you write the same formula twice, one day the two copies disagree, and your slides contradict your own dashboard. With one shared implementation, the numbers on screen and the numbers in our report **can never drift apart**.

But fast technology means nothing if the data going in is wrong — and this dataset had a trap hidden inside it.

Numbers you must know cold

Figure	Value
Raw file	273 MB, 1,048,585 rows
Chunk size	200,000 rows × 6 chunks
Cleaning runtime	~15 seconds
Working table	1.77 million rows
Filter + recompute time	0.06 seconds
Pages / chart types	7 pages, ~17 chart types

The stack, one line each

Tool	Job
pandas	cleaning and aggregation
Parquet (PyArrow)	fast, compressed storage of clean data
Streamlit	Python → web dashboard
Plotly	interactive charts with tooltips
matplotlib + seaborn	static charts for notebook and report

Questions you own

"Why Parquet instead of keeping the CSV?"

Parquet is compressed, typed, and column-based. Reading it is roughly 20 times faster than re-parsing 273 MB of text every time.

"Why not a database?"

One dataset, refreshed in batch, fully processed in 15 seconds by pandas. A database would add a moving part without removing one.

"Why Streamlit and not Power BI or Tableau?"

Our cleaning rules and scoring model are Python code. Streamlit calls that code directly. In a BI tool we would have to rebuild the score in a different language — that is exactly the "two copies drift apart" problem.

"Could this scale to a real product?"

Yes — the pipeline already handles a million rows in 15 seconds. What's missing is hosting, user accounts, and a scheduled data refresh. Not the analytics.

Rehearsal checklist

- ☐ Timed at **3:00 or under**
- ☐ You can say "1.77 million rows, 0.06 seconds" from memory
- ☐ You can explain Parquet in one plain sentence
- ☐ Handover line practised — it sets up the whole next section